libraries from scratch. You'll get the most out of this chapter if you use these opportunities to put the book down and spend some time investigating possible approaches. When you design your own libraries, you won't be handed a nicely chosen sequence of type signatures to fill in with implementations. You'll have to make the decisions about what types and combinators you need, and a goal in part 2 of this book has been to prepare you for doing this on your own. As always, if you get stuck on one of the exercises or want some more ideas, you can keep reading or consult the answers. It may also be a good idea to do these exercises with another person, or compare notes with other readers online.

Parser combinators versus parser generators

You might be familiar with *parser generator* libraries like Yacc (http://mng.bz/w3zZ) or similar libraries in other languages (for instance, ANTLR in Java: http://mng.bz/aj8K). These libraries *generate* code for a parser based on a specification of the grammar. This approach works fine and can be quite efficient, but comes with all the usual problems of code generation—the libraries produce as their output a monolithic chunk of code that's difficult to debug. It's also difficult to reuse fragments of logic, since we can't introduce new combinators or helper functions to abstract over common patterns in our parsers.

In a parser combinator library, parsers are just ordinary first-class values. Reusing parsing logic is trivial, and we don't need any sort of external tool separate from our programming language.

9.1 Designing an algebra, first

Recall that we defined *algebra* to mean a collection of functions operating over some data type(s), *along with a set of laws* specifying relationships between these functions. In past chapters, we moved rather fluidly between inventing functions in our algebra, refining the set of functions, and tweaking our data type representations. Laws were somewhat of an afterthought—we worked out the laws only after we had a representation and an API fleshed out. There's nothing wrong with this style of design,[1] but here we'll take a different approach. We'll *start* with the algebra (including its laws) and decide on a representation later. This approach—let's call it *algebraic design*—can be used for any design problem but works particularly well for parsing, because it's easy to imagine what combinators are required for parsing different kinds of inputs.[2] This lets us keep an eye on the concrete goal even as we defer deciding on a representation.

[1] For more about different functional design approaches, see the chapter notes for this chapter.

[2] As we'll see, there's a connection between algebras for parsers and the classes of languages (regular, context-free, context-sensitive) studied by computer science.

There are many different kinds of parsing libraries.[3] Ours will be designed for expressiveness (we'd like to be able to parse arbitrary grammars), speed, and good error reporting. This last point is important. Whenever we run a parser on input that it doesn't expect—which can happen if the input is malformed—it should generate a parse error. If there are parse errors, we want to be able to point out exactly where the error is in the input and accurately indicate its cause. Error reporting is often an afterthought in parsing libraries, but we'll make sure we give careful attention to it.

OK, let's begin. For simplicity and for speed, our library will create parsers that operate on strings as input.[4] We need to pick some parsing tasks to help us discover a good algebra for our parsers. What should we look at first? Something practical like parsing an email address, JSON, or HTML? No! These tasks can come later. A good and simple domain to start with is parsing various combinations of repeated letters and gibberish words like `"abracadabra"` and `"abba"`. As silly as this sounds, we've seen before how simple examples like this help us ignore extraneous details and focus on the essence of the problem.

So let's start with the simplest of parsers, one that recognizes the single character input `'a'`. As in past chapters, we can just *invent* a combinator for the task, `char`:

```
def char(c: Char): Parser[Char]
```

What have we done here? We've conjured up a type, `Parser`, which is parameterized on a single parameter indicating the *result type* of the `Parser`. That is, running a parser shouldn't simply yield a yes/no response—if it succeeds, we want to get a *result* that has some useful type, and if it fails, we expect *information about the failure*. The `char('a')` parser will succeed only if the input is exactly the character `'a'` and it will return that same character `'a'` as its result.

This talk of "running a parser" makes it clear our algebra needs to be extended somehow to support that. Let's invent another function for it:

```
def run[A](p: Parser[A])(input: String): Either[ParseError,A]
```

Wait a minute; what is `ParseError`? It's another type we just conjured into existence! At this point, we don't care about the representation of `ParseError`, or `Parser` for that matter. We're in the process of specifying an *interface* that happens to make use of two types whose representation or implementation details we choose to remain ignorant of for now. Let's make this explicit with a `trait`:

---

[3] There's even a parser combinator library in Scala's standard libraries. As in the previous chapter, we're deriving our own library from first principles partially for pedagogical purposes, and to further encourage the idea that no library is authoritative. The standard library's parser combinators don't really satisfy our goals of providing speed and good error reporting (see the chapter notes for some additional discussion).

[4] This is certainly a simplifying design choice. We can make the parsing library more generic, at some cost. See the chapter notes for more discussion.

```
trait Parsers[ParseError, Parser[+_]] {

  def run[A](p: Parser[A])(input: String): Either[ParseError,A]
  def char(c: Char): Parser[Char]
}
```

`Parser` is a type parameter that itself is a covariant type constructor

Here the `Parser` type constructor is applied to `Char`.

What's with the funny `Parser[+_]` type argument? It's not too important for right now, but that's Scala's syntax for a type parameter that is itself a type constructor.[5] Making `ParseError` a type argument lets the `Parsers` interface work for any representation of `ParseError`, and making `Parser[+_]` a type parameter means that the interface works for any representation of `Parser`. The underscore just means that whatever `Parser` is, it expects one type argument to represent the type of the result, as in `Parser[Char]`. This code will compile as it is. We don't need to pick a representation for `ParseError` or `Parser`, and we can continue placing additional combinators in the body of this trait.

Our `char` function should satisfy an obvious law—for any `Char`, `c`,

```
run(char(c))(c.toString) == Right(c)
```

Let's continue. We can recognize the single character `'a'`, but what if we want to recognize the string `"abracadabra"`? We don't have a way of recognizing entire strings right now, so let's add that:

```
def string(s: String): Parser[String]
```

Likewise, this should satisfy an obvious law—for any `String`, `s`,

```
run(string(s))(s) == Right(s)
```

What if we want to recognize either the string `"abra"` *or* the string `"cadabra"`? We could add a very specialized combinator for it:

```
def orString(s1: String, s2: String): Parser[String]
```

But choosing between two parsers seems like something that would be more generally useful, regardless of their result type, so let's go ahead and make this polymorphic:

```
def or[A](s1: Parser[A], s2: Parser[A]): Parser[A]
```

We expect that `or(string("abra"),string("cadabra"))` will succeed whenever either `string` parser succeeds:

```
run(or(string("abra"),string("cadabra")))("abra") == Right("abra")
run(or(string("abra"),string("cadabra")))("cadabra") == Right("cadabra")
```

Incidentally, we can give this `or` combinator nice infix syntax like `s1 | s2` or alternately `s1 or s2`, using implicits like we did in chapter 7.

[5] We'll say much more about this in the next few chapters.

Listing 9.1 Adding infix syntax to parsers

```
trait Parsers[ParseError, Parser[+_]] { self =>
  ...
  def or[A](s1: Parser[A], s2: Parser[A]): Parser[A]
  implicit def string(s: String): Parser[String]
  implicit def operators[A](p: Parser[A]) = ParserOps[A](p)
  implicit def asStringParser[A](a: A)(implicit f: A => Parser[String]):
    ParserOps[String] = ParserOps(f(a))

  case class ParserOps[A](p: Parser[A]) {
    def |[B>:A](p2: Parser[B]): Parser[B] = self.or(p,p2)
    def or[B>:A](p2: => Parser[B]): Parser[B] = self.or(p,p2)
  }
}
```

This introduces the name `self` to refer to this `Parsers` instance; it's used later in `ParserOps`.

Use `self` to explicitly disambiguate reference to the `or` method on the `trait`.

We've also made `string` an implicit conversion and added another implicit `asStringParser`. With these two functions, Scala will automatically promote a `String` to a `Parser`, and we get infix operators for any type that can be converted to a `Parser[String]`. So given `val P: Parsers`, we can then `import P._` to let us write expressions like `"abra" | "cadabra"` to create parsers. This will work for *all* implementations of `Parsers`. Other binary operators or methods can be added to the body of `ParserOps`. We'll follow the discipline of keeping the primary definition directly in `Parsers` and delegating in `ParserOps` to this primary definition. See the code for this chapter for more examples. We'll use the `a | b` syntax liberally throughout the rest of this chapter to mean `or(a,b)`.

We can now recognize various strings, but we don't have a way of talking about repetition. For instance, how would we recognize three repetitions of our `"abra" | "cadabra"` parser? Once again, let's add a combinator for it:[6]

```
def listOfN[A](n: Int, p: Parser[A]): Parser[List[A]]
```

We made `listOfN` parametric in the choice of `A`, since it doesn't seem like it should care whether we have a `Parser[String]`, a `Parser[Char]`, or some other type of parser. Here are some examples of what we expect from `listOfN`:

```
run(listOfN(3, "ab" | "cad"))("ababcad") == Right("ababcad")
run(listOfN(3, "ab" | "cad"))("cadabab") == Right("cadabab")
run(listOfN(3, "ab" | "cad"))("ababab") == Right("ababab")
```

At this point, we've just been collecting up required combinators, but we haven't tried to refine our algebra into a minimal set of primitives, and we haven't talked much about more general laws. We'll start doing this next, but rather than give the game away, we'll ask you to examine a few more simple use cases yourself and try to design a

[6] This should remind you of a similar function we wrote in the previous chapter.

minimal algebra with associated laws. This should be a challenging exercise, but enjoy struggling with it and see what you can come up with.

Here are additional parsing tasks to consider, along with some guiding questions:

- A `Parser[Int]` that recognizes zero or more `'a'` characters, and whose result value is the number of `'a'` characters it has seen. For instance, given `"aa"`, the parser results in `2`; given `""` or `"b123"` (a string not starting with `'a'`), it results in `0`; and so on.
- A `Parser[Int]` that recognizes *one* or more `'a'` characters, and whose result value is the number of `'a'` characters it has seen. (Is this defined somehow in terms of the same combinators as the parser for `'a'` repeated zero or more times?) The parser should fail when given a string without a starting `'a'`. How would you like to handle error reporting in this case? Could the API support giving an explicit message like `"Expected one or more 'a'"` in the case of failure?
- A parser that recognizes zero or more `'a'`, followed by one or more `'b'`, and which results in the pair of counts of characters seen. For instance, given `"bbb"`, we get `(0,3)`, given `"aaaab"`, we get `(4,1)`, and so on.

And additional considerations:

- If we're trying to parse a sequence of zero or more `"a"` and are only interested in the number of characters seen, it seems inefficient to have to build up, say, a `List[Char]` only to throw it away and extract the length. Could something be done about this?
- Are the various forms of repetition primitive in our algebra, or could they be defined in terms of something simpler?
- We introduced a type `ParseError` earlier, but so far we haven't chosen any functions for the API of `ParseError` and our algebra doesn't have any way of letting the programmer control what errors are reported. This seems like a limitation, given that we'd like meaningful error messages from our parsers. Can you do something about it?
- Does `a | b` mean the same thing as `b | a`? This is a choice you get to make. What are the consequences if the answer is yes? What about if the answer is no?
- Does `a | (b | c)` mean the same thing as `(a | b) | c`? If yes, is this a primitive law for your algebra, or is it implied by something simpler?
- Try to come up with a set of laws to specify your algebra. You don't necessarily need the laws to be complete; just write down some laws that you expect should hold for any `Parsers` implementation.

Spend some time coming up with combinators and possible laws based on this guidance. When you feel stuck or at a good stopping point, then continue by reading the next section, which walks through one possible design.